[MIDWAY REPORT] IS THE GAP AN INTERFACE GAP? AN AGENT TOOL INTERFACES FOR DATA CATALOGS

Alex Feng

1 PROBLEM STATEMENT (PROBLEM SPECIFICATION)

Maintainers of a dbt-backed lakehouse need catalog metadata (ownership, grain, filters, lifecycle state) that stays trustworthy as models evolve, but it drifts from the code—annotations "can become outdated or incorrect" as data models evolve (Lakafosis et al., 2025)—and the evidence for a correction is scattered across catalog metadata, SQL, lineage, tests, and the data. We are building a *catalog steward* for them with two triggers over one workflow: *remediation* of existing hygiene debt and *change review* of a proposed contribution. Our question: **does a specialized evidence-gathering, risk-aware action, and verification workflow produce more complete, grounded, and safe maintenance outcomes than static checks or a generic coding agent with the same raw repository and warehouse access?**

2 METHODOLOGY

2.1 ENVIRONMENT & DATA

Environment. The environment is a repository-local dbt Core project (dbt Labs, 2026) backed by DuckDB (Raasveldt & Mühleisen, 2019) through the dbt-duckdb adapter. Three small synthetic seed tables (5 customers, 8 orders, 10 order items) feed a staging model and two aggregates (`customer_orders`, `daily_revenue`), with schema documentation, dbt tests, and a JSON catalog snapshot; a compact `catalog/overrides.json` sidecar injects controlled metadata changes. It is a reproducible test bed, not evidence about production warehouses. Each episode copies the clean base, applies a debt or change overlay, generates the diff, and builds the project, so every condition starts from the same state.

Hygiene contract. A versioned contract (`design/hygiene_contract.yaml`, v0.1.0, validated by a JSON Schema) bounds the task to five issue types: missing required metadata, SQL/metadata drift, grain or join drift, unsupported reliability claims, and lifecycle drift. Each type names its required evidence, allowed and forbidden action classes, and verification. Limits: 20 assets, two lineage hops, five findings, and 10,000 probed rows; deleting assets, changing access policies, mutating a live catalog, inventing business semantics, and unbounded queries are forbidden.

Evaluation data and ground truth. The pilot suite has ten hand-constructed, versioned episodes (`data/episodes/manifest.yaml`): four *remediation* cases (a stale revenue-filter description, wrong customer-order grain, a missing owner, and a legacy copy still marked active and canonical), four *flawed change* cases (a changed revenue filter, a changed grain without updated grain metadata or tests, a new undocumented model, and a rename without a lifecycle transition), and two *clean controls* (a documented new model and a documented revenue metric). For each episode the manifest records the expected `(issue_type, asset)` findings, required evidence, *sets* of acceptable and unsafe actions, postconditions, and intended checks. Action sets let an escalation count as correct where business meaning is uncertain (e.g. in the filter-drift case, proposing a description fix is acceptable but editing the SQL to match stale prose is unsafe); clean controls make spurious findings measurable.

Success criteria and metrics. The scorer (`midpoint-score-v1`) matches findings to the manifest exactly at the `(issue_type, asset)` level. An episode *succeeds* only with no missed and no extra finding, no action outside the issue type's allowed classes, no recorded verification failure, and a complete run. It also records precision, recall, action and evidence validity, patches, escalations, cost, and turns. Known limits: evidence validity checks that cited IDs resolve, not that the evidence supports the claim; the action score checks permitted classes, not the episode-specific acceptable and unsafe actions; and postconditions are not checked. We therefore audit every trace by hand before reading score gaps as quality or safety gaps.

2.2 METHODS

Shared workflow and evidence tools. Both triggers run the same inspect–plan–act–verify state machine; change review additionally collects the diff. For the scoped assets, the steward builds an evidence bundle from catalog metadata, the dbt schema and manifest, model SQL with facts parsed by SQLGlot (Mao, 2026) (sources, filters, joins, aggregates, grouping, grain candidates), one-hop lineage, and bounded profiles. Following the view that purpose-built agent–computer interfaces matter (Yang et al., 2024), every evidence item carries a stable content-hashed ID and provenance pointer, and absent metadata is explicit `present: false` evidence. During planning the model may call nine schema-checked, read-only tools (e.g. two-hop lineage, bounded profiles, confined file reads). It returns at most five prioritized findings, each with claim, confidence, impact, evidence, and action.

Risk-aware action policy. Before anything executes, a validator checks the plan schema, in-scope assets, supported issue types, each issue type's required evidence families, action permissions, and that every cited evidence ID resolves. Actions fall into three classes: a *patch* for evidence-backed, reversible technical metadata; a *proposal* that leaves semantic or organizational decisions to a reviewer; and a *report* when no action is justified. Patches touch only five catalog fields in the episode-local override file, never a live catalog.

Verification and ledger. For a patch the steward emits a unified diff, applies it inside the episode, checks that the catalog field now equals the value the model declared and that `dbt parse` succeeds, and otherwise restores the file and records a verification failure. This is weak: `dbt parse` does not read `catalog/`, and the contract's per-issue checks (e.g. comparing metadata to parsed SQL, rerunning a uniqueness probe) do not run yet. Proposals and reports get permission and evidence checks only; an append-only ledger records findings, evidence, actions, and checks.

Conditions. The frozen configuration (`experiments/configs/midpoint_v1.yaml`, v1.2.0) defines three conditions. (1) *Static checks*: required catalog/schema fields plus actual dbt test results, with no SQL parsing or semantic inference. (2) *Generic agent*: the same model, repository, built warehouse, budgets, and output schema, but only four raw operations (list files, read a file, read the diff, bounded read-only SQL) and the issue labels, without the evidence bundle, per-issue rules, or verification. (3) *Steward*: the full workflow above. Both model conditions use `openai/gpt-6-luna` with reasoning effort low, temperature 0, a 4,096-token limit, at most 12 turns, one reply protocol, and one identical repair turn for a final answer that fails validation (prompts `generic-hygiene-v3`, `specialized-steward-v3`). Each episode is one sample (temperature is likely ignored by this reasoning model). Generic-agent patches are plan text only, never applied, while a failed steward verification loses the episode. The comparison tests workflow, evidence, and tools together.

2.3 WHAT DID NOT WORK, AND PIVOTS

From catalog QA to a hygiene steward. The proposal planned a 180-question catalog QA benchmark on a Docker Unity Catalog/Iceberg stack, swept across five harnesses and many models. After feedback that the motivation was narrow and the sweep too ambitious, we pivoted to the maintenance problem underneath, with two headline outcomes (a verified repair and a prevented regression), on a local fixture.

Evaluator defects and a label leak, fixed before any paid run. An audit of the first static run fixed two scorer defects (stale `dbt test` results reused; duplicate findings inflating action rates). A scripted-model preflight of the paid pipeline fixed lost transcripts and costs on failed runs and found an *episode-label leak* through three channels: the steward prompt contained the episode ID (e.g. `rem_daily_revenue_filter_drift`, which names the expected finding), workspace paths that dbt copies into readable artifacts embedded it too, and DuckDB file readers in the generic agent's SQL tool could have read the label manifest itself. Opaque workspace tokens, path rewriting, and disabled DuckDB external access fixed it, and a test asserts that no label reaches either condition's model input.

Protocol repair, and a failure left unfixed. In a paid two-episode smoke run the model copied the wrapper object from the reply-format example, the main source of rejected replies; because the generic agent makes more calls, this penalized it more. We fixed the shared example and error

messages and added the identical repair turn (config 1.2.0); a smoke rerun passed a pre-set gate, and the pilot then ran once. The pilot exposed a new failure—the generic agent batching several tool calls, once with invented results, into one reply—which we left unfixed under a pre-committed no-iteration rule, to avoid tuning prompts on the evaluation set.

3 Results (Summary of Progress)

Completed work and setup. Everything in Section 2 is implemented and covered by 93 deterministic tests that make no model calls. The pilot ran each condition once on the ten episodes from fresh fixtures under config 1.2.0: 30 runs, 0 errors, about 8.5 minutes, $0.0172 of model spend ($0.0244 for the whole project). Four of the 20 model records (the two demo episodes in both model conditions) are exact cache replays of the 1.2.0 smoke rerun, not new samples. We audited every trace by hand (`analysis.md`).

Table 1: Episode successes by condition and category (remediation, flawed change, clean control), finding counts, and micro precision/recall; an errored run counts its expected findings as missed. Source: `experiments/results/midpoint_v1_2_pilot/summary.json`.

Condition	Remed.	Flawed	Clean	All	TP/FP/FN	Micro P/R	Errors	Cost ($)
Static checks	1/4	1/4	2/2	4/10	4/0/6	1.00/0.40	0	0.00
Generic agent	2/4	1/4	2/2	5/10	5/4/5	0.56/0.50	0	0.01
Steward	2/4	3/4	0/2	5/10	9/6/1	0.60/0.90	0	0.01

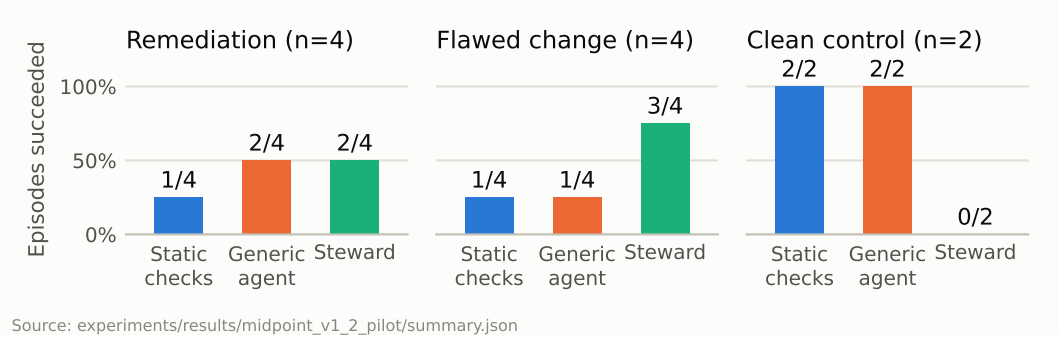


Figure 1: Episode success by condition and category, one run per episode. The steward leads on flawed changes and fails both clean controls, which static checks and the generic agent pass.

Interpretation. On ten author-built fixtures with one run each, the steward matched 9 of 10 expected issues, against 5 for the generic agent and 4 for static checks (Table 1), and every steward claim cited real evidence. It still only tied the generic agent on episode success (5/10 each; static 4/10) because it over-reported: six extra findings, three of them real but unlabelled fixture gaps, including both clean-control flags (an undocumented seed, and a model description that omits a newly documented metric). The generic agent's score is fragile the other way: one success is a pass in which it read no files; one failure rested on tool output it invented, which still passed the ID-based evidence check; and one run approved the refund regression without reading anything, an unsafe action the scorer cannot see. Under alternative label decisions the audit's sensitivity range is steward 7/10, generic 4/10, and static 3/10; this shows how much the headline rests on labels and is not a rescoring.

What the comparison does not isolate. The conditions differ by more than prompting: only the steward gets a pre-collected evidence bundle, the per-issue action contract, and a validator that enforces it, so its 14 model calls against the generic agent's 65, and its unsafe-action rate, are not like-for-like. On the wrong-grain episode both agents made the same grain patch; the steward avoided the penalty only because it labelled the problem SQL/metadata drift, where patches are allowed. Its three applied patches were correct on manual inspection, but its verification checks only the written value and `dbt parse`, not the manifest's semantic checks. Protocol scaffolding also weighs more

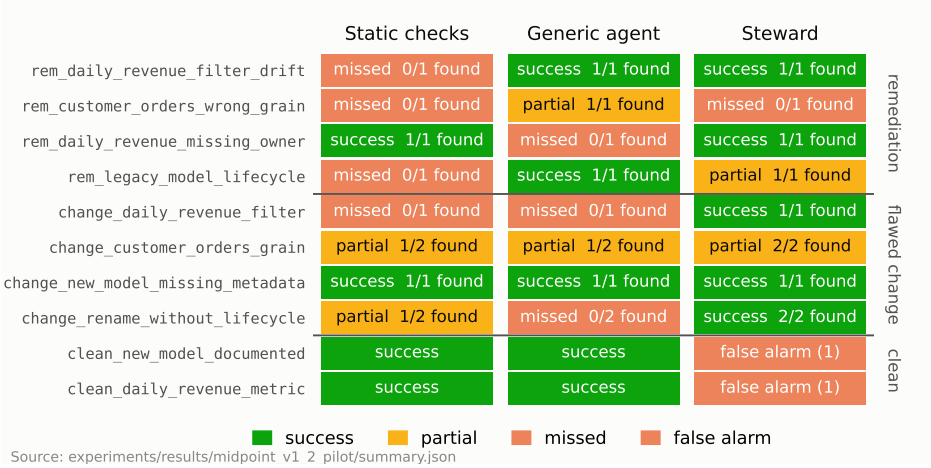


Figure 2: Per-episode outcome. *Partial*: at least one expected finding matched but the episode failed (a miss, an extra finding, or a disallowed action). *Missed*: no expected finding matched. *False alarm*: any finding on a clean control, true or not (both steward flags are real, unlabelled fixture gaps).

on the generic agent: 7 of its 10 runs had a rejected reply, against 2 of 10 for the steward. We make no general or statistical claim.

4 Assets

Repository: `https://github.com/alfenn/agentic-catalog-interface` (Feng, 2026): `README.md` (setup, reproduction), `src/catalog_steward/`, `data/` (fixture, episodes), and `experiments/results/midpoint_v1_2_pilot/` (pilot results, audit `analysis.md`). Midpoint demo video: **[VIDEO URL TO BE ADDED]**.

5 Timeline (Summary of Progress: Next Steps and Risks)

Milestones. Course deadlines are Demo Day on December 9 and the final 8-page report on December 11; intermediate dates are our targets.

- *By November 13:* a new frozen episode version: documented seeds, truly clean controls, an issue-type precedence rule checked independently of the model's label, and human review of the AI-built labels.
- *By November 20:* postcondition-based success and episode-level unsafe-action checks; real per-issue patch verification; a batching-robust protocol.
- *By December 4:* a checklist baseline, tool-versus-workflow ablations, and growth toward 30–50 audited episodes.
- *December 9 and 11:* live demo of both modes; final report.

Obstacles and concerns. No credential or pipeline blocker remains. The main concern is validity: label choices and protocol behaviour move results as much as the workflow, so fixes go into a new frozen version, not a rescoring.

Risks and mitigations. *Scope growth:* a frozen five-type contract with hard limits. *Ambiguous ground truth* (seen in both clean controls): documented seeds, a precedence rule, human label review. *Unsupported claims* (one invented-evidence run passed): claim–evidence support checks and manual audit. *Unfair generic baseline:* checklist baseline, bundle ablation, batching-robust protocol. *Warning overload* (six steward extras): clean controls, a five-finding cap, flagging only gaps a change introduces. *Small pilot:* per-episode reporting, more episodes.

REFERENCES

dbt Labs. dbt Core, version 1.12.6. GitHub repository `dbt-labs/dbt-core` (Apache-2.0), 2026. URL `https://github.com/dbt-labs/dbt-core`.

Alex Feng. agentic-catalog-interface: Catalog steward midpoint prototype, hygiene contract, episode manifest, evaluation configuration, and pilot results. GitHub repository; paths in the text are relative to its root, 2026. URL `https://github.com/alfenn/agentic-catalog-interface`.

Vasileios Lakafosis, Hannes Roth, Benjamin Renard, Wenlong Dong, Zhonghu Gao, and Dave Kurtzberg. How Meta understands data at scale. Engineering at Meta Blog, April 28, 2025, 2025. URL `https://engineering.fb.com/2025/04/28/security/how-meta-understands-data-at-scale/`.

Toby Mao. SQLGlot: An easily customizable SQL parser and transpiler, version 29.0.1. GitHub repository `tobymao/sqlglot` (MIT), 2026. URL `https://github.com/tobymao/sqlglot`.

Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

A AI-USE DISCLOSURE STATEMENT

Generative AI agents did most of the implementation and writing at this stage. OpenAI Codex agents implemented the prototype (environment setup, agent harness, hygiene contract, evidence tools, steward workflow, and baseline runners), built the dbt/DuckDB fixtures and the ten evaluation episodes and their labels, wrote the evaluation and scoring code, ran and audited the static pilot, and drafted the README and an initial version of this report. Claude (Anthropic) agents then audited, tested, fixed, and finished the work, including the preflight audit of the paid-run pipeline and its fixes, running the paid smoke runs and the three-condition pilot, the manual audit of every pilot trace, typesetting this report in the ICLR template, the references, the script that generates the charts and results table from saved results, and the revision of this report around the audit. Because AI agents constructed the episodes and labels as well as the system under test, the ground truth reflects their design choices and needs independent human review. **[AUTHOR TO COMPLETE: describe the human verification actually performed, e.g., which code, episode labels, results, and report claims the author reviewed.]** AI tools are not authors; the author takes full responsibility for the content of this work.